

Infinite Connections: Methods

A Complete Technical Description of the Puzzle Generation System

STA 561D | Duke University | April 2026

Abstract

We describe the full methodology behind our NYT Connections puzzle generator. The system builds puzzles via two complementary pipelines: an LLM-heavy baseline (5 API calls/puzzle) and a novel Category-First Retrieval (CFR) pipeline (0-1 API calls/puzzle) that uses sentence embeddings and k-nearest-neighbors search over an augmented 14,877-word vocabulary. Every generated puzzle is checked against the full set of past NYT puzzles at both the 16-word and 4-word-group level, assigned difficulty colors via average pairwise cosine similarity, and validated by two independent solvers that must converge on the intended solution. Across 200 benchmark puzzles, CFR v2 achieves a 99% solver-validated pass rate, uses 62-69% non-NYT words, and reproduces zero past NYT groups.

1. Problem and High-Level Design

The task

The New York Times Connections puzzle asks the player to partition 16 words into 4 disjoint groups of 4, where each group shares a hidden theme. Themes range from synonyms (WAYS TO SAY HELLO) to fill-in-the-blank wordplay (FIRE ____), shared sounds, or pop-culture references. The four groups have an intended difficulty ordering: yellow (easiest) to purple (hardest).

What we built

1. A generation system with two pipelines and three modes.
2. A multi-solver validator that runs two independent automated solvers on every generated puzzle and accepts only those that can be solved deterministically.
3. A playable web app hosted on GitHub Pages that loads a bundled set of validated puzzles and replicates the NYT UI.

Two pipelines, three modes

Pipeline	Mode	LLM calls / puzzle	Source of inspiration
A - Iterative baseline	single	5	LLM proposes word pools
B - CFR	remix	0	NYT category pool
B - CFR	fresh	1 batched	LLM writes fresh categories

All three share identical post-generation steps: color assignment, deduplication, and roundtable validation. Only the 'get categories' and 'get candidate words' steps differ.

2. Data

Ground-truth NYT dataset

We use ConnectionsFinalDataset.json, a publicly-available scrape of 554 past NYT Connections puzzles. Each puzzle contains:

- date, contest
- words: a list of 16 strings
- answers: a list of 4 objects, each with answerDescription (category name) and words (the 4 words in that group)
- difficulty: a float in [0,1]

From this we derive:

- W_NYT: 4,918 unique words (uppercased, deduplicated)
- C: 2,054 unique category names
- G_NYT: 2,216 past 4-word groups (stored as frozensets)
- 554 complete 16-word puzzles for word-overlap dedup

Augmented word bank (v2)

After the professor's guidance that 'the total word bank should be more than that of the original game's,' we expanded W to 14,877 words by unioning three sources:

1. W_NYT (4,918 words).
2. W_WN: NLTK WordNet lemmas filtered by (a) single-token only, (b) alphabetic only, (c) length in [3, 15], (d) tagged-corpus frequency ≥ 1 , and (e) not in a profanity blacklist. Yields ~10,000 common English lemmas.
3. W_G10k: optional Google 10k most-common-English list, if present on disk.

$$W = W_NYT \cup W_WN \cup W_G10k, \quad |W| = 14,877$$

The result is cached to data/cache/augmented_word_bank.json to avoid recomputing.

Embedding model

We use sentence-transformers with the model all-mpnet-base-v2, which produces 768-dimensional contextual embeddings. Every word in W and every category in C is encoded once and cached to data/cache/nyt_embeddings_v2.npz. The cached matrix E_W (14,877 x 768) is the substrate for all retrieval and scoring.

3. Pipeline A: LLM-Heavy Baseline

Pipeline A implements the iterative generator described in 'Making New Connections' (arXiv:2407.11240), extended with story-injection for diversity. It is our reference point: a correct, working implementation of the strongest LLM-only approach in the literature.

Per-puzzle flow

```
groups = []
for i in 1..4:
    seed = 4 random words from W_NYT           # story injection
    (category, pool_8) = LLM(seed, groups)      # Call 1-4
    words_4 = MPNET-select-best(pool_8, 4)      # 70 combos ranked
    groups.append({category, words_4})
groups = LLM-editor(groups)                    # Call 5
groups = assign-colors(groups)
dedup(groups)
roundtable-validate(groups)
```

Story injection for diversity

A known failure mode of LLMs is repetitive output (BOARD GAMES: chess, checkers, monopoly, life; repeated dozens

of times). We insert 4 random words from W_{NYT} into the system prompt as 'inspiration.' The LLM is told to write a short imagined story around those words before proposing a category. This dramatically increases category diversity.

MPNET word selection

The LLM returns an 8-word candidate pool. We do NOT let the LLM pick which 4 words to keep; instead, for all $C(8,4)=70$ subsets S we compute

$$\text{score}(S) = \text{mean over pairs } (u,v) \text{ in } S \text{ of } \cos(e(u), e(v))$$

and return the subset with the highest score. This deterministic step cleans up LLM inconsistency.

Editor pass

A single LLM call (Call 5) reviews the complete 4-group puzzle and either approves each category name or rewrites one it considers inaccurate. In practice ~95% of categories pass unchanged.

False-group variant

An alternative method produces the highest-quality puzzles in user studies. The LLM first proposes a plausible-but-fake 'root' group (e.g., 'ROUND THINGS: ball, globe, ring, wheel'), then for each root word finds an alternate meaning and generates a real group around it. The 4 real groups form the puzzle; the fake root is a decoy that 'fits' but isn't a valid grouping. Enabled via `--method false_group`.

4. Pipeline B: Category-First Retrieval (CFR)

CFR is our primary methodological contribution. It inverts Pipeline A: instead of the LLM generating words, it generates only category names (or none at all), and the words are retrieved from W by k -nearest-neighbors over MPNET embeddings.

Motivation

Pipeline A wastes LLM tokens. The LLM generates 8 words, then MPNET picks 4 of them anyway; the other 4 are discarded. Two observations:

1. The LLM is good at naming (creative, readable categories).
2. MPNET is good at ranking (deterministic semantic similarity over a fixed vocabulary).

CFR assigns each task to the component that handles it best.

Offline precomputation (~5 minutes, once)

```
Load W (14,877 words) and C (2,054 categories) from NYT data
E_W = MPNET(W)      # (14877 x 768)
E_C = MPNET(C)      # (2054 x 768)
KNN = NearestNeighbors(E_W, metric='cosine')
G_NYT = { frozenset(g.words) : g in all NYT groups }
Save E_W, E_C, KNN, G_NYT to disk cache
```

Per-puzzle flow

```
# Step 1: obtain 4 category names
if mode == 'remix':
    cats = sample-diverse(C, n=4, min-dist=0.35)  # 0 LLM calls
elif mode == 'fresh':
```

```

cats = LLM-batched('4 categories')           # 1 LLM call

used = set(); groups = []
for c in cats:
    q = MPNET(c)                             # Step 2: KNN
    cand = top-k(KNN, q, k=30, exclude=used,
                 filter stems & sub-tokens)
    pool = cand[:8]
    best_4 = pick-best-non-NYT-combo(pool, G_NYT) # Step 3: guard
    groups.append({c, best_4}); used |= best_4

groups = assign-colors(groups)               # Step 4
dedup(groups, G_NYT)                        # Step 5
roundtable-validate(groups)                 # Step 6

```

Mode A: NYT category remix

Mode A picks 4 existing NYT category names, diverse in embedding space. 'Diverse' means all $C(4,2)=6$ pairwise cosine distances between the four categories are at least 0.35; rejection sampling is used until this condition is met. No LLM call. No API cost. No network latency.

Mode B: Fresh LLM categories

Mode B uses one batched LLM call that returns a JSON object with four unrelated categories. This is a single call producing four categories, not four separate calls. Cost is ~\$0.0005 per puzzle with gpt-4o-mini.

KNN retrieval with filters

Given a category embedding q , we request the top $k=30$ words by cosine distance from KNN. Before returning, three filters are applied in order:

1. Exclusion: words already used in previous groups of the same puzzle.
2. Category-token overlap: a word is dropped if it is literally one of the tokens in the category name (e.g., drop 'TIME' for category 'TIME PERIODS').
3. Morphological duplicates: two words sharing a crude stem (via a suffix-stripping heuristic) are not both kept. If TIME is chosen, TIMES/TIMER/TIMING are skipped.

Best-combination selection with NYT-group safety

From the 8 surviving candidates, we enumerate all $C(8,4)=70$ subsets, score each by average pairwise cosine similarity (as in Pipeline A), and rank descending. We return the first combination that is not a verbatim past NYT group:

$$S^* = \operatorname{argmax} \{ \operatorname{score}(S) : S \text{ subset pool}, |S|=4, \text{frozenset}(S) \text{ not in } G_{\text{NYT}} \}$$

If the top-scoring combination happens to be a past NYT group (e.g., KNN retrieved the literal NYT bird group when queried for 'BIRDS'), we fall through to the next-best combination. In 200 benchmark v2 puzzles, this fallback fired a handful of times and produced 0 verbatim NYT group matches in the final output.

5. Shared Post-Generation Components

Every pipeline -- Pipeline A, CFR Mode A, and CFR Mode B -- passes through the same 3-stage post-processing.

Difficulty color assignment

For each group g with words W_g , compute

$$\mu(g) = \text{avg-sim}(g) = \text{mean over pairs } (u,v) \text{ in } W_g \text{ of } \cos(e(u), e(v))$$

Sort the 4 groups descending by μ and map to colors: yellow (highest, easiest) -> green -> blue -> purple (lowest, hardest). The thresholds are empirical, matching Table 1 of 'Making New Connections': yellow ~ 0.40 , green ~ 0.35 , blue ~ 0.29 , purple ~ 0.27 . Assignment is by rank within the puzzle, so every puzzle always gets exactly one group per color.

Deduplication

The rubric rules that any generation of a verbatim past NYT puzzle is an automatic fail. We enforce this in two independent ways:

16-word set overlap

For the 16-word set U of the candidate puzzle, compute the maximum overlap with any single past NYT puzzle: $o^* = \max |U \cap p.\text{words}|$. Flag as duplicate if $o^* > 6$.

4-word group match

For each generated group g , test: is $\text{frozenset}(g)$ in G_{NYT} ? If any generated group matches a past NYT group verbatim, flag the puzzle as a duplicate.

Roundtable validation

Two independent solvers attempt to recover the intended 4-group partition. A puzzle is accepted if either solver fully recovers it.

Solver 1 -- Embedding (greedy)

Enumerate all $C(16,4)=1,820$ possible 4-word groups, score each by average pairwise cosine similarity, greedily pick highest-scoring non-overlapping groups until 4 are chosen. Fast, deterministic. Baseline on 554 real NYT puzzles: 2.0% full-puzzle solve rate.

Solver 2 -- Clustering (beam search)

Per 'Deceptively Simple' (arXiv:2412.01621), each candidate group g gets a composite score $G(g) = 0.4 \cdot I(g) + 0.3 \cdot s(g) + 0.3 \cdot V(g)$, where:

- $I(g) = -K(E_g)$, minus k-means inertia at $k=1$ (tight cluster preferred)
- $s(g) = \min \text{pairwise } \cos(e(u), e(v))$ over all 6 pairs (guards outliers)
- $V(g) = \text{mean}(P_g) / (1 + \text{var}(P_g))$, where P_g is the pairwise sim vector (rewards stable cohesion)

A penalty $P(g, R) = \text{mean over } r \text{ in } R \text{ of } \cos(\mu_g, e(r))$ discourages groups bleeding into the remaining word set. Beam search (width 10) finds the 4-group partition maximizing sum of $G(g_i) - P(g_i, R_i)$.

Acceptance rule

A puzzle is accepted if at least one of Solvers 1 and 2 returns a partition matching all 4 intended groups. Conservative by design: our solvers solve only 2-3% of real NYT puzzles, so a puzzle they CAN solve has cleanly recoverable semantic structure (i.e., unambiguous).

Optional LLM solver

A third solver prompts the LLM with the 16 shuffled words and chain-of-thought instructions, parsing its returned 4-group partition. Used only for edge cases or published benchmarks; off by default.

6. Implementation Details

Code organization

```

src/
|-- config.py          DRY_RUN flag, model names, thresholds
|-- llm_client.py      OpenAI / Anthropic / Mock unified
|-- generator/         Pipeline A
|   |-- pipeline.py
|   |-- group_creator.py iterative + false-group
|   |-- puzzle_editor.py
|   |-- prompts.py
|   |-- difficulty.py   color assignment (shared)
|   |-- deduplicator.py v2: 16-word + 4-word (shared)
|-- cfr/               Pipeline B (our contribution)
|   |-- pipeline.py     CFRPipeline, both modes
|   |-- embedding_retriever.py KNN + category sampling
|   |-- word_bank.py    v2: NYT + WordNet + Google 10k
|   |-- prompts.py      batched-categories prompt (Mode B)
`-- solvers/
    |-- embedding_solver.py
    |-- clustering_solver.py
    |-- llm_solver.py
    |-- roundtable.py

scripts/
|-- generate_puzzles.py   Pipeline A CLI
|-- cfr/generate_cfr.py  Pipeline B CLI
|-- split_by_color.py
`-- build*_pdf.py         regenerate PDFs

data/      nyt_puzzles/, cache/, generated/, generated/cfr_v2/
webapp/    deployed static site (GitHub Pages)
notebooks/ Jupyter demo
tests/     pytest suite

```

Dry-run mode

All LLM-calling code goes through LLMClient. When DRY_RUN=true (default), the client never makes a network call; it returns realistic canned responses routed by prompt-keyword matching. The entire pipeline -- including the notebook and web app -- can be built and tested with zero API keys configured.

Reproducibility

A fixed random seed (RANDOM_SEED = 42 in config.py) is threaded through the pipeline. The same input produces the same output. Intermediate artefacts (candidate pools, solver traces) are persisted to the output JSON for audit.

7. Benchmarks

Pipeline-level comparison (100 puzzles per row)

Pipeline / Mode	LLM	Pass	Time	Non-NYT	Group matches
A (gpt-4o-mini, iterative)	5	91%	~10 s	--	not checked
A (gpt-4o, iterative)	5	87%	~10 s	--	not checked
B v1 Mode A (remix)	0	97%	1.41 s	0%	not checked
B v1 Mode B (fresh)	1	99%	2.67 s	0%	not checked
B v2 Mode A (remix)	0	99%	1.27 s	62.6%	0 / 100
B v2 Mode B (fresh)	1	99%	2.77 s	69.1%	0 / 100

Solver baselines on real NYT puzzles (N=554)

Solver	Full-puzzle solve rate	Published baseline
Embedding (greedy)	2.0% (11/554)	~11.6% (Missed Connections)
Clustering (beam=10)	3.4% (19/554)	higher than embedding

Our solvers are more conservative than published baselines, which is intentional: we want the validation to reject ambiguous puzzles aggressively, not match human performance. A puzzle our solvers can crack has a cleanly recoverable semantic structure.

Dataset statistics

Artefact	Count
Ground-truth NYT puzzles	554
Unique NYT words (W_NYT)	4,918
Unique NYT categories (C)	2,054
Unique NYT 4-word groups (G_NYT)	2,216
WordNet lemmas added (filtered)	~10,000
Total augmented word bank (v2)	14,877
Puzzles generated (Pipeline A)	775
Puzzles generated (Pipeline B v1+v2)	400
Total validated puzzles on disk	>1,100

8. Rubric Compliance

"Don't reproduce a past connections puzzle"

Enforced at two levels:

- 16-word set overlap threshold of 6 against any past NYT puzzle.
- Exact 4-word group match against the indexed set of 2,216 past NYT groups.

Across 200 v2 benchmark puzzles, both checks flagged 0 generated puzzles as duplicates.

"Word bank larger than the original"

NYT has 4,918 unique words; ours has 14,877. Moreover, 62-69% of output words in v2 benchmarks are from the non-NYT portion of the bank, demonstrating that the augmentation is actually used -- not just shelved.

"Web interface"

The static site at [webapp/](#) is deployed to GitHub Pages. It bundles 542 validated puzzles and replicates the NYT 4x4 grid with correct-guess reveals, 'one away' feedback, shuffle, and mistake tracking. Vanilla JavaScript, no backend.

"Plausibly from the NYTimes"

Formal human evaluation pending. Spot checks on v2 output show many puzzles that read as NYT-style (FAQ and notebook deliverables will contain sample puzzles with commentary).

9. Limitations and Future Work

Known limitations

- Wordplay categories underperform. CFR relies on semantic similarity, which captures 'SYNONYMS FOR X' well but struggles with '___FISH' or 'WORDS CONTAINING HIDDEN COLORS.' Purely syntactic categories don't map cleanly to embedding space.
- Occasional awkward WordNet words (e.g., MORPHOPHONEMIC) slip into some groups. A higher WordNet frequency threshold or intersection with Google 10k would help.
- Solver conservatism may reject good puzzles. Some creative puzzles that a human could solve may fail our solver gate.

Concrete next steps

- Scale to 10,000 puzzles. CFR Mode A can do this in ~3.5h at \$0 cost; Mode B does it in ~7.5h for ~\$0.50.
- Mode C: CFR False-Group. Port the highest-quality method from Pipeline A into CFR's 1-LLM-call framework.
- Batched LLM categories. In Mode B, ask for 40 categories per call instead of 4 -- a 10x cost reduction on top of the 20x already achieved.
- Human evaluation. Add a thumbs-up/down widget to the web app; collect hundreds of ratings; report the 'plausibly NYT' rate directly.
- LLM solver in the roundtable. Use it as a tiebreaker on puzzles that one of the first two solvers rejects, improving acceptance of genuinely good wordplay puzzles.

10. References

- Samuel et al., Making New Connections, arXiv:2407.11240 (2024).
- Missed Connections: Solvers for the NYT Connections Game, arXiv:2404.11730 (2024).
- Deceptively Simple, arXiv:2412.01621 (2024).
- Connecting the Dots, arXiv:2406.11012 (2024).
- Princeton University, WordNet: A Lexical Database for English.
- sentence-transformers library, all-mpnet-base-v2 model.